

# ADVANCED ALGORITHM MINI PROJECT

## **CRICKET ELIMINATION PROBLEM**

using Edmonds-Karp version of Ford-Fulkerson Algorithm

MEGHANA N-PES2UG20CS519

# PROBLEM STATEMENT

- Decision making and Prediction plays major role on every sector. May it be Elections, Games or company target.
- Given the standings in a sports league at some point during the season, determine which teams have been mathematically eliminated from winning their division.
- Here we predict the sequence in which the cricket teams are eliminated. And there winning probability and number of matches a particular team can play with other teams



# MOTIVATION

- As almost all Indians are cricket fans they will be very curious about the champion of the particular cricket league, So we decided to try to predict the winning probability of the teams and elimination probability and other things



# YOUR APPROACH

- Using Edmonds-Karp version of Ford-Fulkerson Algorithm we can eliminate the option based on given previous data and constraints
- Here we predict
  - >which team eliminates first
  - >number of games they can play
  - >number of times they face each other in the left out games
  - >winning probability.



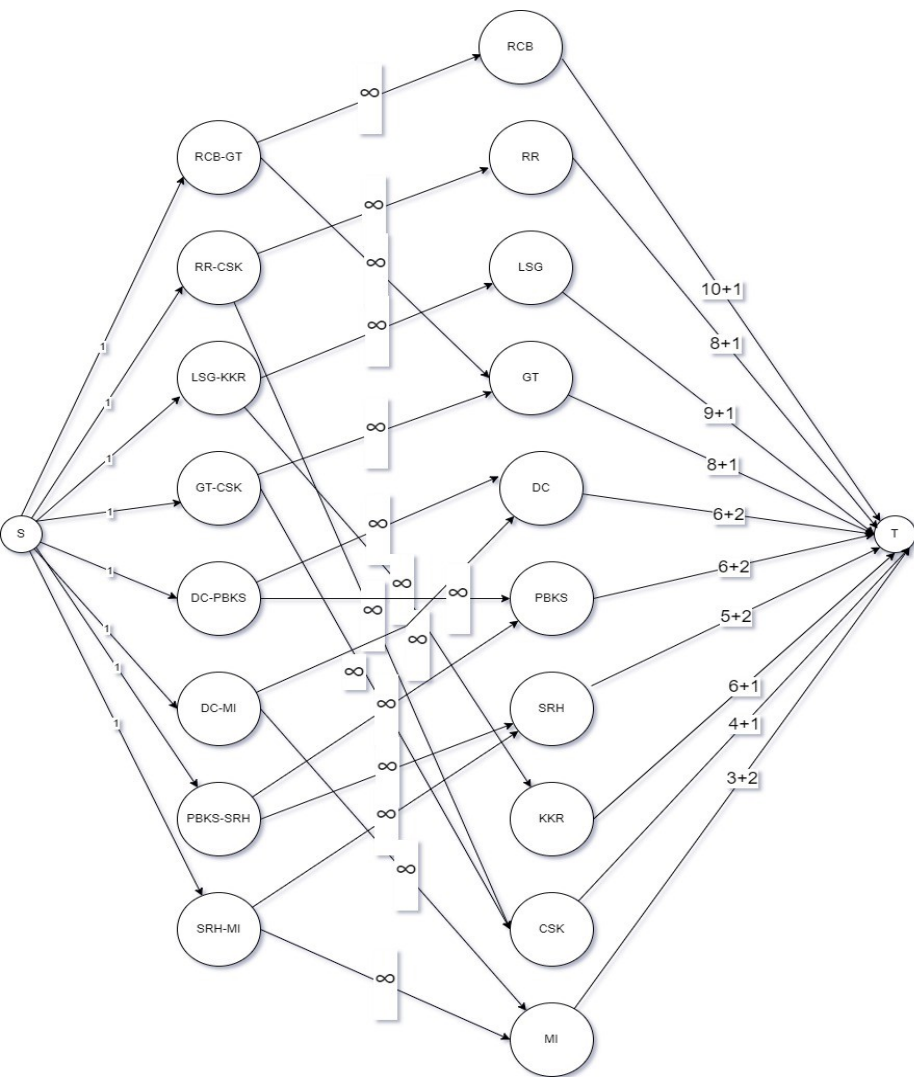
# SYSTEM ARCHITECTURE, DESIGN, IMPLEMENTATION HIGHLIGHTS

- **The Cricket elimination problem.** In the Cricket elimination Problem, there is a league consisting of  $N$  teams. At some point during the season, team  $i$  has  $W[i]$  wins and  $g[i][j]$  games left to play against team  $j$ . A team is eliminated if it cannot possibly finish the season in first place or tied for first place. The goal is to determine exactly which teams are eliminated. The problem is not as easy as many sports writers would have you believe, in part because the answer depends not only on the number of games won and left to play, but also on the schedule of remaining games
- **A maximum flow formulation.** We now solve the Cricket elimination Problem by reducing it to the maximum flow problem. To check whether or not one particular team  $x$  is eliminated, we create a network and solve a maximum flow problem in it. In the network, feasible integral flows correspond to outcomes of the remaining schedule. There are vertices corresponding to teams (other than team  $x$ ) and to remaining divisional games not involving team  $x$ . Intuitively, each unit of flow in the network corresponds to a remaining game. As it flows through the network, it passes from a game node, say between teams  $i$  and  $j$ , then through one of the team nodes  $i$  or  $j$ , classifying this game as being won by that team.



More precisely, our graph includes the following edges and capacities. We connect an artificial source  $s$  to each game node  $i$ - $j$  and set its capacity to  $g[i][j]$ . If a flow uses all  $g[i][j]$  units of capacity on this edge, then we think of all these games as being played, and the wins are distributed between team nodes  $i$  and  $j$ . We connect each game node  $i$ - $j$  with the two opposing teams to ensure that one of the two teams earns a win. We do not need to restrict the amount of flow on such edges. Finally, we connect each team node to an artificial sink  $t$ . Team  $x$  can win as many as  $w[x] + r[x]$  games, so to prevent team  $i$  from winning more than that many games in total, we include an edge from team node  $i$  to the sink with capacity  $w[x] + r[x] - w[i]$ . If the max flow in the network below saturates all arcs leaving  $s$  then this corresponds to assigning winners to all of the remaining games in such a way that no team wins more games than  $x$ . If the max flow does not saturate all arcs leaving  $s$  then there is no scenario in which team  $x$  wins the division.





our graph includes the following edges and capacities. We connect an artificial source  $s$  to each game node  $i-j$  and sets its capacity to  $g[i][j]$ . If a flow uses all  $g[i][j]$  units of capacity on this edge, then we think of all these games as being played, and the wins are distributed between team nodes  $i$  and  $j$ . We connect each game node  $i-j$  with the two opposing teams to ensure that one of the two teams earns a win. We do not need to restrict the amount of flow on such edges. Finally, we connect each team node to an artificial sink  $t$ . Team  $x$  can win as many as  $w[x] + r[x]$  games, so to prevent team  $i$  from winning more than that many games in total, we include an edge from team node  $i$  to the sink with capacity  $w[x] + r[x] - w[i]$ . If the max flow in the network below saturates all arcs leaving  $s$  then this corresponds to assigning winners to all of the remaining games in such a way that no team wins more games than  $x$ . If the max flow does not saturate all arcs leaving  $s$  then there is no scenario in which team  $x$  wins the division.



# ANALYSIS OF THE RESULT

- 1. Lets take an example that RCB won 9 matches lost 2 matches and left with 1 match. And MI won 6 matches lost 4 matches and left with 2 matches. MI is mathematically eliminated since it can finish with at most 8 wins and RCB already has 9 wins. This is the simplest reason for elimination.

| TEAM  | Wins | Loss | Left |
|-------|------|------|------|
| ----- |      |      |      |
| RCB   | 9    | 2    | 1    |
| GT    | 7    | 4    | 1    |
| RR    | 7    | 3    | 2    |
| MI    | 6    | 4    | 2    |

However, there can be more reasons. For example, RR is mathematically eliminated. It is season with as many as 9 appears to be enough would require RCB to remaining games.

